

Linux pour l'admin cloud

Jour 2 — Faire tourner des services

Bloc CL-LINUX (jour 7 du cursus) — processus, systemd, paquets et journaux :

le quotidien d'un serveur qui travaille pour vous

Objectifs de la journée

À la fin de cette journée, vous serez capable de :

- Lister et comprendre les **processus** d'un serveur (`ps aux`, `pstree`, `top`/`htop`).
- Arrêter proprement un processus avec les **signaux** (`SIGTERM` avant `SIGKILL`).
- Piloter un **service systemd** : `status`, `start`, `stop`, `restart`, `enable`.
- **Écrire votre propre unité systemd** et la faire démarrer au boot.
- Installer et gérer des logiciels avec **APT** (`update`, `upgrade`, `install`, `remove`).
- Lire les **journaux** avec `journalctl` et `/var/log`, et comprendre `logrotate`.

Le réflexe à ancrer aujourd'hui, celui de tous les dépannages :

`systemctl status` → `journalctl -u` → **corriger** → `daemon-reload` → `restart`.

Plan de la journée

1. **Les processus** — ce qui tourne vraiment sur la machine (~1 h 15).
2. **Les services systemd** — piloter nginx comme un pro (~1 h 30) —  démo 2-1,  exercice 2-1.
3. **Écrire une unité** — votre premier service maison (~45 min) —  exercice 2-2.
4. **Les paquets APT** — installer, mettre à jour, nettoyer (~1 h).
5. **Les journaux** — journalctl, /var/log, logrotate (~1 h 15).
6. Récap + quiz de fin de journée (~30 min).

Environnement : votre VM **Ubuntu Server 24.04** d'hier (`multipass shell srv-linux`).

Toutes les commandes du jour sont rejouables telles quelles dedans.

1. Les processus

Ce qui tourne vraiment sur la machine

Qu'est-ce qu'un processus ?

Un **processus** = un programme **en cours d'exécution** : du code chargé en mémoire, identifié par un numéro unique, le **PID** (Process ID).

- Chaque processus a un **parent** (PPID) : c'est le parent qui l'a lancé.
- Quand vous tapez `ls`, votre shell (`bash`) crée un processus enfant, attend sa fin, récupère son **code retour**.
- Un processus consomme du **CPU** et de la **mémoire** : les deux ressources que vous surveillerez (et paierez !) sur toutes vos machines cloud.

```
echo $$      # le PID de votre shell
sleep 300 &  # lancer un processus en arrière-plan
ps           # les processus de VOTRE terminal seulement
```

Sur un serveur, des dizaines de processus tournent **sans aucun terminal** : ce sont eux qui nous intéressent.

ps aux — la photo complète du système

```
ps aux          # TOUS les processus, de tous les utilisateurs
ps aux | grep ssh # filtrer (réflexe quotidien)
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	1.2	22560	12800	?	Ss	09:02	0:01	/sbin/init
ubuntu	1234	0.3	0.5	8272	5120	pts/0	Ss	09:15	0:00	-bash
ubuntu	1290	0.0	0.1	6104	1920	pts/0	S	09:18	0:00	sleep 300

Colonnes à savoir lire :

- **USER** : qui a lancé le processus — **root** doit rester l'exception.
- **PID** : le numéro pour agir dessus (**kill**, **systemctl**...).
- **%CPU / %MEM** : consommation — vos premiers suspects quand « c'est lent ».
- **STAT** : **S** en sommeil, **R** en cours, **Z** zombie, **T** stoppé.
- **COMMAND** : la commande exacte, avec ses arguments.

pstree — qui a lancé qui ?

Les processus forment un **arbre généalogique**, du PID 1 jusqu'à votre **ls** :

```
pstree -p          # l'arbre complet avec les PID
pstree -p ubuntu   # seulement les processus de l'utilisateur ubuntu
```

```
systemd(1)─sshd(820)─sshd(1210)─bash(1234)─pstree(1301)
          │
          └─nginx(901)─nginx(902)
                  │
                  └─nginx(903)
          │
          └─cron(650)
          │
          └─systemd-journald(320)
```

Deux lectures essentielles :

- **Tout descend de `systemd` (PID 1)** — on en reparle dans la section 2.
- Votre commande `pstree` est l'arrière-petit-fils de `sshd` : c'est le chemin exact de votre connexion. Sur EC2, ce sera le même arbre, au PID près.

top et htop — la supervision en direct

```
top    # installé partout, austère mais universel
htop   # version confortable (déjà installée via notre cloud-init)
```

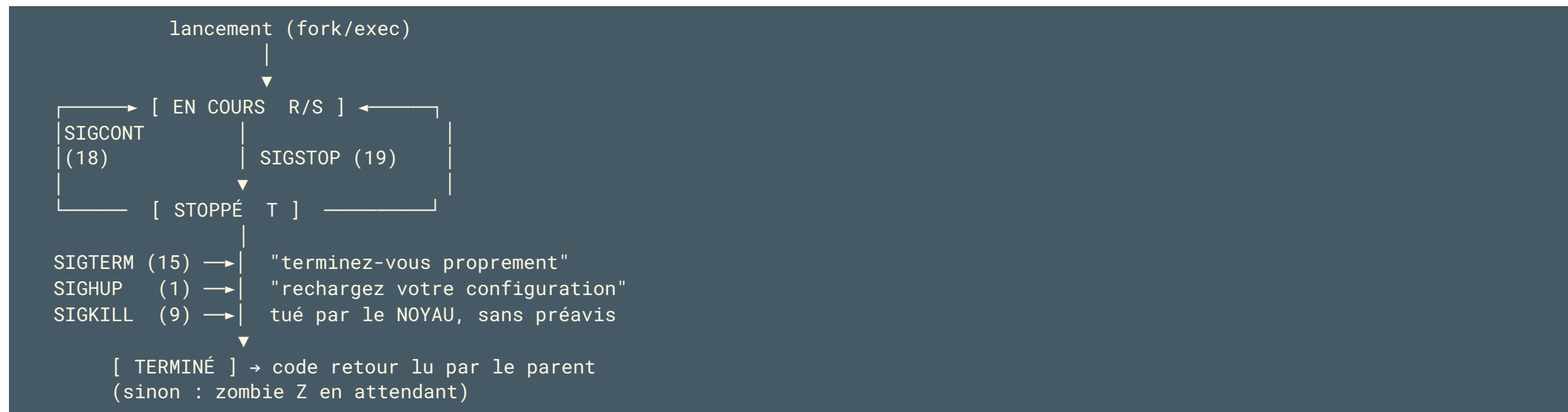
À lire dans l'en-tête de `top` :

- **load average** : `0.15, 0.10, 0.05` = charge moyenne sur 1, 5 et 15 min.
Repère : une charge $\approx$ nombre de CPU = machine pleine (2 vCPU $\rightarrow$ 2.0).
- **MiB Mem** : `free` qui fond n'est pas grave, `avail Mem` est le vrai indicateur.
- La liste est triée par **%CPU** décroissant : le gourmand est en haut.

Touches utiles (les deux outils) : `M` tri mémoire, `P` tri CPU, `k` envoyer un signal, `q` quitter. Dans `htop` : `F5` bascule en vue arbre (comme `pstree`).

Anticipation cloud : ces chiffres (CPU, mémoire, charge) sont exactement les **métriques CloudWatch** que vous grapherez au bloc CL-AWS.

Le cycle de vie d'un processus et les signaux



Un **signal** = un petit message du système à un processus.

SIGTERM est **interceptable** (le processus nettoie),

SIGKILL ne l'est **jamais** (le noyau tranche).

kill et pkill — envoyer des signaux

`kill` envoie un **signal** (pas forcément mortel !) à un PID :

```
kill 1290      # SIGTERM (15) par défaut : "terminez-vous proprement"
kill -15 1290  # identique, explicite
kill -9 1290   # SIGKILL : le noyau tue, sans nettoyage possible
kill -HUP 901  # SIGHUP : beaucoup de démons rechargent leur config
pkill sleep    # SIGTERM à tous les processus nommés "sleep"
pkill -u alice # tous les processus d'un utilisateur
```

La règle professionnelle : toujours `SIGTERM` d'abord, attendre quelques secondes, `SIGKILL` en **dernier recours** seulement.

Pourquoi ? Avec `kill -9`, le processus ne peut ni fermer ses fichiers, ni terminer ses écritures, ni prévenir ses clients → **données corrompues** possibles. Sur une base de données en production, c'est la catastrophe type.

Premier plan, arrière-plan : `&`, `jobs`, `fg`, `bg`,

`nohup`

Votre terminal ne fait qu'**une chose à la fois**... sauf si vous demandez :

```
sleep 600 &           # & : lancer directement en arrière-plan
jobs                  # lister les tâches de CE terminal ([1] Running...)
fg %1                 # ramener la tâche 1 au premier plan
# Ctrl+Z              # suspendre la tâche au premier plan (SIGSTOP)
bg %1                 # la relancer en arrière-plan
nohup ./long-calcul.sh & # survit à la fermeture du terminal (nohup.out)
```

- `Ctrl+C` envoie SIGINT (interrompre), `Ctrl+Z` envoie SIGSTOP (suspendre).
- Sans `nohup`, fermer le terminal SSH tue vos tâches d'arrière-plan (SIGHUP).

Mais retenez surtout ceci : sur un serveur, `nohup ./mon-app &` est du bricolage — pas de redémarrage au boot, pas de relance en cas de crash, pas de journaux. La bonne réponse s'appelle **systemd**. Transition parfaite.

2. Les services systemd

Le chef d'orchestre de votre serveur

systemd : le PID 1, parent de tout



systemd est le premier processus lancé par le noyau. Il démarre, surveille et arrête tous les **services** – et il journalise tout.

Processus vs service : la nuance qui change tout

	Processus « nu » (<code>nohup ... &</code>)	Service systemd
Démarre au boot	non	oui (<code>enable</code>)
Relancé s'il plante	non	oui (<code>Restart=on-failure</code>)
Journaux centralisés	non (fichier au petit bonheur)	oui (<code>journalctl -u</code>)
Arrêt propre	à vous de trouver le PID	<code>systemctl stop</code>
Identité dédiée	celle du lanceur	<code>User=</code> dans l'unité

Un **service** = un processus (ou un groupe) **pris en charge par systemd** : décrit dans un fichier **unité** (`.service`), supervisé de bout en bout.

Rappel cloud : sur EC2 ou une VM Azure, votre application tournera **exactement comme ça** — une unité systemd. C'est le standard de toutes les distributions serveur modernes (Ubuntu, Debian, RHEL, Amazon Linux).

systemctl status – la première commande de tout diagnostic

```
sudo apt update && sudo apt install -y nginx # notre cobaye du jour
systemctl status nginx
```

```
• nginx.service - A high performance web server and a reverse proxy server
  Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
  Active: active (running) since Thu 2026-07-02 09:30:12 UTC; 2min ago
    Main PID: 901 (nginx)
      Tasks: 3 (limit: 4557)
     Memory: 4.8M
        CPU: 24ms
    CGroup: /system.slice/nginx.service
            └─901 "nginx: master process /usr/sbin/nginx ..."
               └─902 "nginx: worker process"
```

Quatre lignes à lire, toujours dans cet ordre : **Loaded** (l'unité existe-t-elle, est-elle `enabled` ?), **Active** (`running` / `failed` / `inactive`), **Main PID** (le processus réel), et les **dernières lignes de journal** dessous.

Piloter un service : start, stop, restart, reload

```
sudo systemctl stop nginx      # arrêter (SIGTERM propre au processus)
sudo systemctl start nginx     # démarrer maintenant
sudo systemctl restart nginx   # stop + start (coupure brève)
sudo systemctl reload nginx    # recharger la config SANS couper (SIGHUP)
```

- `status` se lance **sans sudo** ; toute action qui **modifie** exige `sudo`.
- `reload` n'existe que si le service le supporte (nginx : oui) – préférez-le à `restart` en production : **zéro coupure** pour les utilisateurs.
- Vérifiez toujours après action :

```
systemctl status nginx        # Active: active (running) ?
curl -I http://localhost     # HTTP/1.1 200 OK = le service répond
```

Réflexe : une action → une vérification. Jamais « ça a dû marcher ».

Au boot ou pas : enable, disable, --failed

start démarre **maintenant** ; au prochain reboot, tout est perdu.

enable programme le démarrage **à chaque boot** (et ne démarre rien maintenant) :

```
sudo systemctl enable nginx      # démarrera à chaque boot
sudo systemctl enable --now nginx # enable + start en une commande
sudo systemctl disable nginx     # ne démarrera plus au boot
systemctl is-enabled nginx       # enabled / disabled
systemctl is-active nginx        # active / inactive / failed
```

Et LA commande de prise en main d'un serveur inconnu :

```
systemctl --failed              # toutes les unités en échec
systemctl list-units --type=service --state=running # ce qui tourne
```

Sur une VM cloud fraîchement démarrée, **systemctl --failed** vide = bon signe.
C'est la première commande à taper quand « le serveur ne marche plus ».

Types d'unités et cibles (targets)

systemd ne gère pas que des services. Une **unité** = un objet géré, typé par son suffixe :

Type	Rôle	Exemple
<code>.service</code>	un programme supervisé	<code>nginx.service</code> , <code>ssh.service</code>
<code>.timer</code>	déclencher une unité à heure fixe	<code>apt-daily.timer</code> (vu demain)
<code>.target</code>	un groupe d'unités, une étape du boot	<code>multi-user.target</code>
<code>.mount</code>	un point de montage	<code>boot-efi.mount</code>
<code>.socket</code>	activation à la connexion	<code>ssh.socket</code>

```
systemctl list-units --type=target # les cibles actives
systemctl get-default             # multi-user.target sur un serveur
```

À retenir : `multi-user.target` = « serveur prêt, mode normal sans interface graphique ». C'est à cette cible qu'on **accroche** ses services — vous l'écrirez tout à l'heure dans `WantedBy=`.

LA méthode de diagnostic (à connaître par cœur)

Un service ne marche pas ? **Toujours la même séquence**, dans cet ordre :

```
systemctl status monservice      # 1. constat : failed ? depuis quand ?
journalctl -u monservice -n 50   # 2. les 50 dernières lignes de journal
sudo nano /etc/systemd/system/monservice.service # 3. corriger la cause
sudo systemctl daemon-reload     # 4. si l'unité a été modifiée
sudo systemctl restart monservice # 5. relancer
systemctl status monservice      # 6. vérifier : active (running)
```

- L'erreur est **presque toujours écrite en clair** dans le journal (étape 2).
- `daemon-reload` : systemd garde les unités **en cache** — tant que vous ne le lancez pas, il exécute **l'ancienne version** du fichier. Piège n°1 du métier.

Cette séquence, vous la rejouerez à l'exercice 2-1, au mini-TP de J4, au CL-TP1 noté... et pendant toute votre carrière. **Apprenez-la aujourd'hui.**



Démo 2-1 — Un service de A à Z (30 min)

Fichier : `demos/03-cl-linux/demo-2-1-service-systemd.md`

Au programme, en direct sur la VM :

1. **Observer** nginx : `systemctl status`, `stop`/`start`, `enable`,
`journalctl -u nginx` — relier chaque ligne à ce qu'on vient de voir.
2. **Écrire** une unité minimaliste `bonjour.service` : un script bash qui logge la date en boucle, pris en charge par systemd.
3. **Casser exprès** l'unité (faute de frappe dans `ExecStart`) pour dérouler la méthode de diagnostic : `status` → `journalctl -u` → corriger → `daemon-reload` → `restart`.

Objectif : voir le cycle de vie complet d'un service **avant** de vous lancer — gardez la méthode sous les yeux, vous en aurez besoin dans 30 minutes.

Exercice 2-1 – Un service qui ne démarre pas (45 min)

Fichier : `exercices/03-cl-linux/exercice-2-1-service-en-panne.md`

Mise en situation : vous arrivez sur un serveur où `horodatage.service` refuse de démarrer (un bloc de mise en place à copier-coller installe la panne).

- **Objectif** : diagnostiquer puis réparer le service en appliquant la méthode — `systemctl status` → `journalctl -u horodatage -n 50` → corriger → `daemon-reload` → `enable --now`.
- Deux anomalies distinctes se cachent dans l'unité : le journal vous dira tout.
- Terminé quand : `systemctl status horodatage` affiche `active (running)` et survit à un `sudo reboot`.

Durée : 45 min — 30 min en autonomie (indices dans l'énoncé), 15 min de correction commentée. C'est l'exercice le plus « vrai métier » de la semaine.

3. Écrire une unité systemd

Votre premier service maison

Anatomie d'un fichier unité

```
/etc/systemd/system/bonjour.service      <- VOS unités vont ici
```

```
[Unit]                                QUOI et QUAND
Description=Affiche la date           <- texte du status
After=network.target                 <- démarrer après le
                                     réseau

[Service]                             COMMENT
ExecStart=/opt/bonjour/bonjour.sh    <- LA commande
                                     (chemin ABSOLU)
User=ubuntu                          <- identité d'exécution
Restart=on-failure                   <- relance si crash

[Install]                             AU BOOT
WantedBy=multi-user.target           <- accroché à la cible
                                     "serveur prêt"
```

Trois sections, toujours les mêmes : **[Unit]** décrit, **[Service]** exécute, **[Install]** accroche au démarrage.

[Unit] et [Service] : les directives à connaître

[Unit] – l'identité et l'ordonnancement :

- `Description=` : la phrase affichée par `systemctl status`. Soignez-la.
- `After=network.target` : « ne me démarre qu'une fois le réseau prêt » – indispensable pour tout service qui écoute ou se connecte.

[Service] – l'exécution :

- `ExecStart=` : la commande, en **chemin absolu** (`/usr/bin/python3`, pas `python3`) – systemd n'a pas votre `PATH`. Cause n°1 d'échec.
- `User=` : ne tournez **jamais** en root sans raison. Un utilisateur dédié limite les dégâts en cas de faille (fil rouge : **sécuriser**).
- `Restart=on-failure` : systemd relance si le processus sort en erreur – l'**auto-réparation** de base de tout service en production.
- `Environment=CLE=valeur` : variables d'environnement (souvenez-vous de `STOCKLINE_DB` au bloc Python – c'est ici qu'elle se déclarera en J4).

[Install], daemon-reload : le circuit complet

[Install] — `WantedBy=multi-user.target` : « quand le système atteint la cible serveur prêt, démarre-moi ». C'est ce qui donne un sens à `enable`.

Le circuit complet pour créer une unité :

```
sudo nano /etc/systemd/system/bonjour.service    # 1. écrire l'unité
sudo systemctl daemon-reload                    # 2. systemd relit ses fichiers
sudo systemctl enable --now bonjour              # 3. au boot + tout de suite
systemctl status bonjour                        # 4. vérifier : running
journalctl -u bonjour -f                        # 5. regarder vivre le service
```

- `daemon-reload` à **chaque modification** du fichier — sinon systemd applique l'ancienne version et vous cherchez une panne fantôme.
- Vos unités : `/etc/systemd/system/`. Celles des paquets : `/usr/lib/systemd/system/` — on ne touche **jamais** aux secondes.

Sur EC2 : votre app StockLine tournera exactement comme ça — **une unité systemd**, écrite à la main vendredi (J4), puis notée au CL-TP1.



Exercice 2-2 – Une unité pour le script d'inventaire (45 min)

Fichier : `exercices/03-cl-linux/exercice-2-2-unite-inventaire.md`

Retour du script Python d'inventaire du bloc CL-PYTHON (fourni dans l'énoncé) : il écrit hostname, uptime et espace disque dans

`/var/lib/inventaire/rapport.txt`.

- **Objectif** : écrire de zéro une unité `.service` (Type=oneshot, `WantedBy=multi-user.target`) pour que l'inventaire s'exécute **à chaque démarrage** de la VM.
- Validation : `sudo reboot`, puis vérifier que le rapport est régénéré et que `journalctl -u inventaire` raconte l'exécution.
- Bonus : `RemainAfterExit=yes`, puis une deuxième unité qui **échoue** volontairement si le disque dépasse 90 % (revoir les codes retour).

Durée : 45 min. Cette fois, page blanche : le schéma « anatomie d'une unité » est votre plan de construction.

4. Les paquets APT

Installer, mettre à jour, nettoyer

Comment un logiciel arrive sur votre serveur

```
/etc/apt/sources.list.d/ubuntu.sources
(la liste des DÉPÔTS : des serveurs
 web pleins de paquets .deb signés)
|
| 1. sudo apt update      "rafraîchir le CATALOGUE"
|
▼
catalogue local (/var/lib/apt/lists/)
versions disponibles, dépendances
|
| 2. sudo apt install nginx
|
▼
résolution des dépendances → téléchargement des .deb
(nginx a besoin de libssl...) depuis les dépôts
|
▼
installation + configuration + service démarré
```

Un **paquet** = le logiciel + ses fichiers + ses dépendances + ses scripts d'installation. **APT** orchestre tout, depuis des dépôts signés.

apt update vs apt upgrade — le duo à ne plus confondre

```
sudo apt update    # rafraîchit le CATALOGUE (n'installe RIEN)
sudo apt upgrade   # installe les mises à jour disponibles
```

- **update** = « va voir ce que proposent les dépôts » : télécharge les listes, compare, annonce **12 packages can be upgraded**. **Aucun logiciel modifié.**
- **upgrade** = « applique les mises à jour » à partir du catalogue local.
- Un catalogue périmé = APT installe de vieilles versions, ou échoue en 404.

L'ordre canonique, à taper en arrivant sur toute machine :

```
sudo apt update && sudo apt upgrade -y
```

C'est le premier geste sur chaque VM cloud fraîchement lancée (vous l'automatiserez plus tard avec cloud-init, puis avec de l'IaC au bloc CL-IAC). Fil rouge : **automatiser** ce qu'on répète.

Installer, désinstaller, nettoyer

```
sudo apt install -y htop tree      # installer (plusieurs paquets d'un coup)
sudo apt remove tree              # désinstaller (GARDE la configuration)
sudo apt purge tree               # désinstaller + supprimer la config
sudo apt autoremove              # purger les dépendances devenues orphelines
```

- **remove** vs **purge** : **remove** laisse les fichiers de config (dans **/etc**) pour une réinstallation future ; **purge** fait table rase.
- **autoremove** : quand vous retirez nginx, ses bibliothèques installées « pour lui » restent — **autoremove** les identifie et les supprime. À lancer régulièrement : sur un disque cloud de 20 Go, ça compte.
- **-y** répond oui aux confirmations : indispensable dans les scripts, à utiliser en conscience en interactif.

Bonne pratique serveur : **n'installez que le nécessaire**. Chaque paquet en plus = surface d'attaque en plus (fil rouge : **sécuriser**).

Chercher et se renseigner : search, show, list

```
apt search proxy      # chercher dans noms + descriptions (sans sudo)
apt show nginx        # fiche d'identité d'un paquet
apt list --installed  # tout ce qui est installé
apt list --upgradable # ce que upgrade mettrait à jour
```

`apt show nginx` vous répond notamment :

```
Package: nginx
Version: 1.24.0-2ubuntu7.5
Depends: nginx-common, libc6, libssl3t64 ...
Description: small, powerful, scalable web/proxy server
```

- **Version** : celle du dépôt Ubuntu 24.04 (stable, maintenue en sécurité 5 ans — c'est ça, une LTS).
- **Depends** : ce qui sera installé en cascade.

Réflexe avant tout `install` : un `apt show` pour savoir **ce qu'on embarque** — un admin cloud sait ce qui tourne sur ses machines.

Les dépôts, et apt vs apt-get

Où APT trouve-t-il ses dépôts ? Sur Ubuntu 24.04 :

```
cat /etc/apt/sources.list.d/ubuntu.sources # le format moderne (deb822)
```

```
Types: deb
URIs: http://archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main restricted universe multiverse
```

- L'historique `/etc/apt/sources.list` est désormais quasi vide : tout vit dans `/etc/apt/sources.list.d/` (un fichier par source – les dépôts tiers, Docker par exemple, y ajouteront le leur au bloc CL-CONT).

apt vs apt-get : même moteur. `apt` = interface **interactive** moderne (couleurs, barre de progression) ; `apt-get` = interface **stable pour les scripts** (sortie garantie inchangée). En interactif tapez `apt`, dans vos scripts d'automatisation écrivez `apt-get`.

Snap : l'autre système de paquets d'Ubuntu (en une slide)

Ubuntu embarque un **second** gestionnaire, **snap** : paquets auto-contenus (toutes dépendances incluses), mis à jour automatiquement, isolés du système.

```

snap list           # les snaps installés (il y en a de base sur 24.04)
sudo snap install hello-world && hello-world
sudo snap remove hello-world

```

	APT (.deb)	snap
Dépendances	partagées	embarquées (plus lourd)
Mises à jour	quand VOUS le décidez	automatiques
Usage typique serveur	tout l'écosystème	quelques cas (LXD, certbot)

À retenir pour ce cursus : **APT est votre outil de référence sur serveur** ; sachez reconnaître un snap (`snap list`) quand vous auditez une machine. On n'en dira pas plus.

5. Les journaux

Ce que le serveur a à vous dire

journalctl — le journal central de systemd

systemd capture **tout ce que les services écrivent** dans un journal central, interrogé avec `journalctl` :

```
journalctl -u nginx      # tout le journal DU service nginx
journalctl -u nginx -n 50 # les 50 dernières lignes (le réflexe diagnostic)
journalctl -u nginx -f    # suivre EN DIRECT (follow) – Ctrl+C pour sortir
```

```
Jul 02 09:30:12 srv-linux systemd[1]: Starting A high performance web server...
Jul 02 09:30:12 srv-linux systemd[1]: Started nginx.service.
```

- `-u` (unit) : LE filtre que vous utiliserez cent fois — c'est l'étape 2 de la méthode de diagnostic.
- `-f` (follow) : gardez un terminal ouvert dessus pendant que vous testez dans un autre — vous voyez l'erreur **au moment où elle se produit**.
- Sans filtre, `journalctl` affiche tout depuis le boot : inutilisable seul, toujours filtrer.

journalctl — cibler dans le temps et par gravité

```
journalctl -u nginx --since "10 minutes ago"    # fenêtre temporelle
journalctl -u nginx --since "2026-07-02 09:00"    # depuis un instant précis
journalctl -u nginx --since today
journalctl -p err -b                             # toutes les ERREURS depuis le boot
journalctl -b                                     # tout depuis ce boot ; -b -1 = boot précédent
```

- `-p` (priority) filtre par gravité : `emerg > alert > crit > err > warning > notice > info > debug`. `-p err` = err **et plus grave**.
- `-b -1` : précieux quand « la machine a redémarré toute seule cette nuit » — le journal du boot précédent contient souvent la cause.

La combinaison gagnante au quotidien :

```
journalctl -p err --since today    # "qu'est-ce qui va mal aujourd'hui ?"
```

Anticipation cloud : au bloc CL-AWS, l'agent **CloudWatch** enverra ces mêmes journaux dans le cloud pour les consulter sans SSH. La source, c'est ici.

`/var/log` — les fichiers classiques

Avant/à côté du journal systemd, les fichiers texte historiques de `/var/log` :

Fichier	Contenu	Quand le lire
<code>/var/log/syslog</code>	le fourre-tout système	comportement bizarre, timeline
<code>/var/log/auth.log</code>	connexions, sudo, échecs SSH	sécurité : qui entre ?
<code>/var/log/nginx/access.log</code>	chaque requête HTTP reçue	trafic, erreurs 404/500
<code>/var/log/nginx/error.log</code>	erreurs du serveur web	le 502 de vendredi...
<code>/var/log/dpkg.log</code>	installations de paquets	« qui a installé ça ? »

```
sudo tail -f /var/log/auth.log      # les outils d'hier servent tous ici
sudo grep "Failed password" /var/log/auth.log
sudo less /var/log/syslog
```

Sur une VM exposée à Internet, `auth.log` se remplit de tentatives d'intrusion **en quelques minutes** — vous le verrez de vos yeux en J4, et c'est pour ça qu'on durcira SSH (fil rouge : **superviser, sécuriser**).

logrotate — pour que les journaux ne mangent pas le disque

Un journal qui grossit sans limite finit par **remplir le disque** → services qui plantent en cascade. Panne classique n°1 des serveurs cloud oubliés.

logrotate fait « tourner » les fichiers : archive, compresse, supprime.

```
access.log → access.log.1 → access.log.2.gz → ... → supprimé
  (actif)      (hier)          (compressé)      (au-delà de la rétention)
```

```
cat /etc/logrotate.d/nginx      # la politique de rotation de nginx
ls /var/log/nginx/              # constater les .1, .2.gz...
```

Directives lisibles dans `/etc/logrotate.d/nginx` : `daily` (rotation quotidienne), `rotate 14` (garder 14 archives), `compress`, `notifempty`.

Chaque paquet dépose sa politique dans `/etc/logrotate.d/` ; une tâche planifiée exécute logrotate chaque jour. Le journal systemd, lui, gère sa propre limite (`journalctl --disk-usage` pour la voir).

Récap des points clés du jour

- Un **processus** = programme en cours (PID) ; `ps aux`, `pstree`,
`top` / `htop` pour voir ; **SIGTERM d'abord, SIGKILL en dernier recours**.
- Un **service** = processus géré par **systemd (PID 1)** : démarré au boot,
surveillé, relancé, journalisé.
- `systemctl` : `status` / `start` / `stop` / `restart` / `reload` /
`enable --now` / `--failed`.
- Une unité = `[Unit] / [Service] / [Install]` ; `ExecStart` en chemin
absolu, `User=`, `Restart=on-failure`, `WantedBy=multi-user.target`.
- **La méthode** : `status` → `journalctl -u -n 50` → corriger →
`daemon-reload` → `restart` → vérifier.
- APT : `update` (catalogue) ≠ `upgrade` (installe) ; `install` / `remove` /
`purge` / `autoremove` ; dépôts dans `/etc/apt/sources.list.d/`.
- Journaux : `journalctl -u -f --since -p -b`, `/var/log` (syslog,
auth.log, nginx/), **logrotate** pour ne pas remplir le disque.

Sur EC2, tout ceci s'applique à **l'identique** : c'est votre futur quotidien.



Quiz — questions 1 et 2

Question 1

Quelle est la différence entre un processus et un service ?

Question 2

Que fait `kill -9` et pourquoi le réserver en dernier recours ?



Quiz — questions 3 et 4

Question 3

Quelle commande liste les unités systemd en échec ?

Question 4

Quelle est la différence entre `systemctl start` et `systemctl enable` ?



Quiz — questions 5 et 6

Question 5

Dans une unité systemd, à quoi servent `ExecStart` et `Restart=on-failure` ?

Question 6

Quelle est la différence entre `apt update` et `apt upgrade` ?



Quiz — questions 7 et 8

Question 7

Où APT trouve-t-il la liste de ses dépôts ?

Question 8

Quelle commande suit en direct les journaux du service nginx ?



Quiz — questions 9 et 10

Question 9

Quel signal `kill` envoie-t-il par défaut, et que demande-t-il au processus ?

Question 10

À quoi sert `logrotate` ?

Jour 3 — Automatiser : scripting bash

Aujourd'hui vous avez piloté des services **à la main**. Demain, la machine travaille pour vous : scripts bash (variables, arguments, conditions, boucles), grep/awk/sed, planification avec cron et les timers systemd — vous écrirez un vrai script de sauvegarde, celui-là même qui protégera StockLine vendredi.

Développé par Utopios, le spécialiste de la transformation digitale intelligente